

# Design and Analysis of Algorithms

## AIML, 5<sup>TH</sup> SEM

CSEL30503MJqqw

### Module - 1

**Complexity Order Notation: definitions and calculating complexity.**

# Content

- Asymptotic Notations
  - Big - Oh Notation ( $O$ )
  - Big - Omega Notation ( $\Omega$ )
  - Big - Theta Notation ( $\Theta$ )
  - Common Asymptotic Notations and Their Importance
- Recurrence Relation
  - Master Method
  - Substitution Method
  - Recurrence Tree Method

# Asymptotic Notations

Definition: Asymptotic notation of an algorithm is a mathematical representation of its complexity.

Majorly, we use THREE types of Asymptotic Notations and those are as follows...

- Big - Oh ( $O$ )
- Big - Omega ( $\Omega$ )
- Big - Theta ( $\Theta$ )

# Big - Oh Notation (O)

- Big - Oh notation is used to define the upper bound of an algorithm in terms of Time Complexity.
- Big - Oh notation always indicates the maximum time required by an algorithm for all input values.
- Big - Oh notation describes the worst case of an algorithm time complexity.
- Big - Oh Notation can be defined as follows...

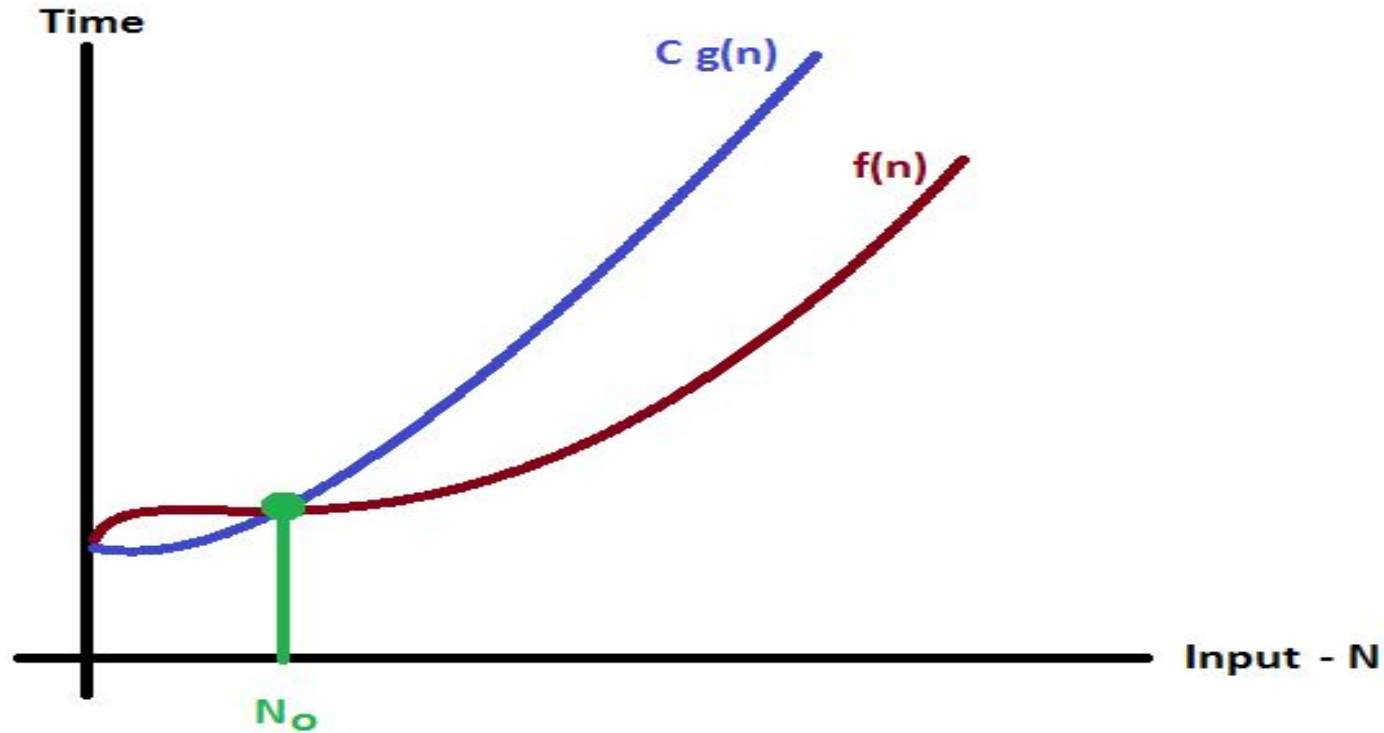
Consider function  $f(n)$  as time complexity of an algorithm and  $g(n)$  is the most significant term.

If  $f(n) \leq C g(n)$  for all  $n \geq n_0$ ,  $C > 0$  and  $n_0 \geq 1$ . Then we can represent  $f(n)$  as  $O(g(n))$ .

$$f(n) = O(g(n))$$

# Big - Oh Notation (O)

- Consider the following graph drawn for the values of  $f(n)$  and  $C g(n)$  for input (N) value on X-Axis and time required is on Y-Axis.



In above graph after a particular input value  $N_0$ , always  $C g(n)$  is greater than  $f(n)$  which indicates the algorithm's upper bound.

# Example: Big - Oh Notation (O)

Consider the following  $f(n)$  and  $g(n)$ ...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent  $f(n)$  as  $O(g(n))$  then it must satisfy  $f(n) \leq C g(n)$  for all values of  $C > 0$  and  $n_0 \geq 1$

$$f(n) \leq C g(n)$$

$$\Rightarrow 3n + 2 \leq C n$$

Above condition is always TRUE for all values of  $C = 4$  and  $n \geq 2$ .

By using Big - Oh notation we can represent the time complexity as follows...

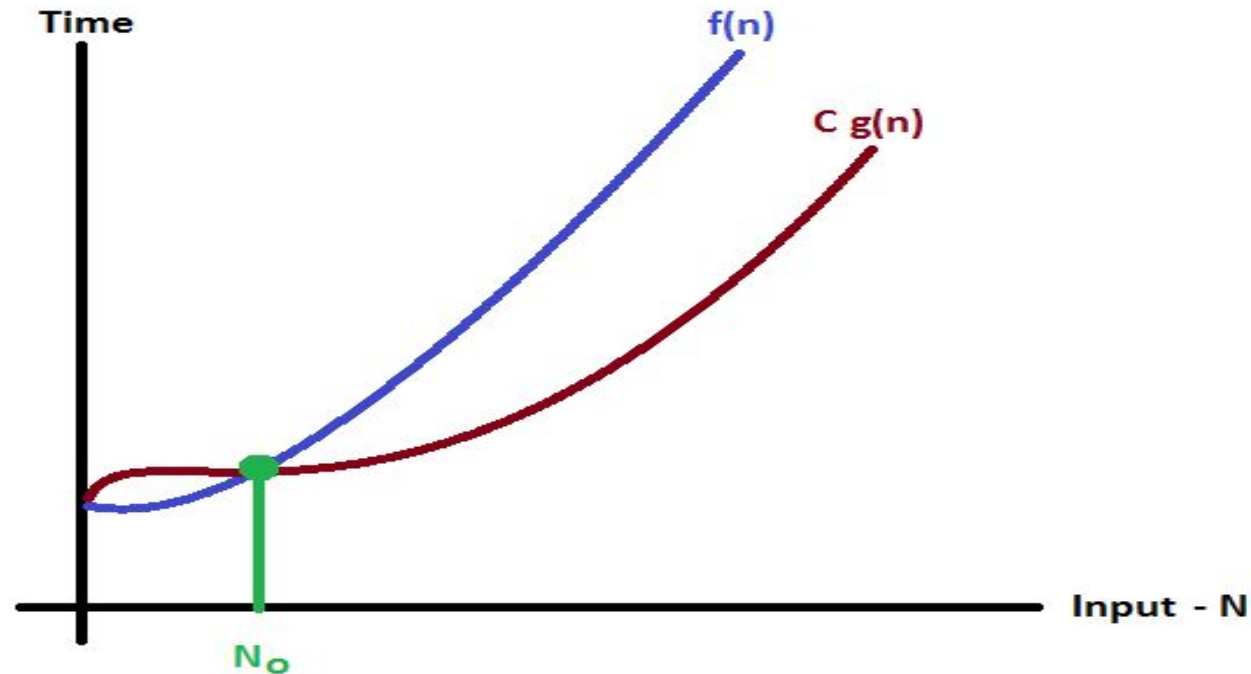
$$3n + 2 = O(n)$$

# Big - Omega Notation ( $\Omega$ )

- Big - Omega notation is used to define the lower bound of an algorithm in terms of Time Complexity.
- That means Big-Omega notation always indicates the minimum time required by an algorithm for all input values. That means Big-Omega notation describes the best case of an algorithm time complexity.
- Big - Omega Notation can be defined as follows...
- Consider function  $f(n)$  as time complexity of an algorithm and  $g(n)$  is the most significant term. If  $f(n) \geq C g(n)$  for all  $n \geq n_0$ ,  $C > 0$  and  $n_0 \geq 1$ . Then we can represent  $f(n)$  as  $\Omega(g(n))$ .
- $f(n) = \Omega(g(n))$

# Big - Omega Notation ( $\Omega$ )

- Consider the following graph drawn for the values of  $f(n)$  and  $C g(n)$  for input ( $n$ ) value on X-Axis and time required is on Y-Axis.



In above graph after a particular input value  $n_0$ , always  $C g(n)$  is less than  $f(n)$  which indicates the algorithm's lower bound.



# Example: Big - Omega Notation ( $\Omega$ )

Consider the following  $f(n)$  and  $g(n)$ ...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent  $f(n)$  as  $\Omega(g(n))$  then it must satisfy  $f(n) \geq C g(n)$  for all values of  $C > 0$  and  $n_0 \geq 1$

$$f(n) \geq C g(n)$$

$$\Rightarrow 3n + 2 \geq C n$$

Above condition is always TRUE for all values of  $C = 1$  and  $n \geq 1$ .

By using Big - Omega notation we can represent the time complexity as follows...

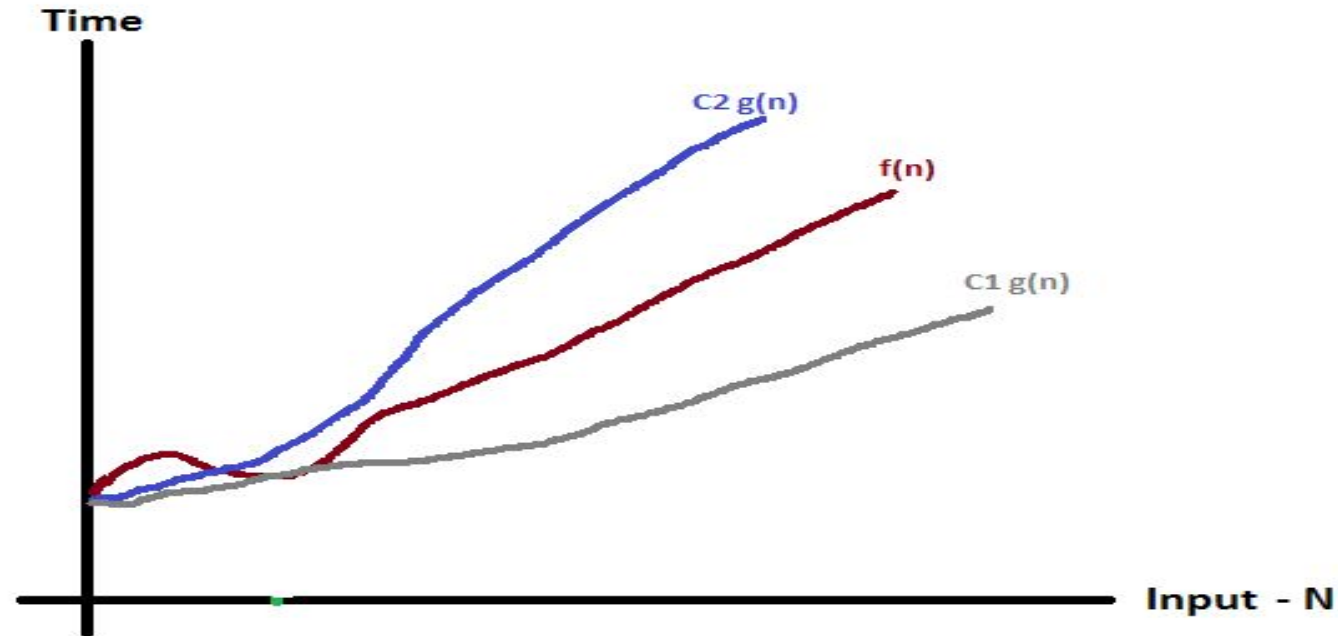
$$3n + 2 = \Omega(n)$$

# Big - Theta Notation ( $\Theta$ )

- Big - Theta notation is used to define the average bound of an algorithm in terms of Time Complexity.
- That means Big - Theta notation always indicates the average time required by an algorithm for all input values. That means Big - Theta notation describes the average case of an algorithm time complexity.
- Big - Theta Notation can be defined as follows...
- Consider function  $f(n)$  as time complexity of an algorithm and  $g(n)$  is the most significant term. If  $C1 g(n) \leq f(n) \leq C2 g(n)$  for all  $n \geq n_0$ ,  $C1 > 0$ ,  $C2 > 0$  and  $n_0 \geq 1$ . Then we can represent  $f(n)$  as  $\Theta(g(n))$ .
- $f(n) = \Theta(g(n))$

# Big - Theta Notation ( $\Theta$ )

- Consider the following graph drawn for the values of  $f(n)$  and  $C g(n)$  for input ( $n$ ) value on X-Axis and time required is on Y-Axis



In above graph after a particular input value  $n_0$ , always  $C1 g(n)$  is less than  $f(n)$  and  $C2 g(n)$  is greater than  $f(n)$  which indicates the algorithm's average bound.

# Example: Big - Theta Notation ( $\Theta$ )

Consider the following  $f(n)$  and  $g(n)$ ...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent  $f(n)$  as  $\Theta(g(n))$  then it must satisfy  $C1 \cdot g(n) \leq f(n) \leq C2 \cdot g(n)$  for all values of  $C1 > 0$ ,  $C2 > 0$  and  $n_0 \geq 1$

$$C1 \cdot g(n) \leq f(n) \leq C2 \cdot g(n)$$

$$\Rightarrow C1 \cdot n \leq 3n + 2 \leq C2 \cdot n$$

Above condition is always TRUE for all values of  $C1 = 1$ ,  $C2 = 4$  and  $n \geq 2$ .

By using Big - Theta notation we can represent the time complexity as follows...

$$3n + 2 = \Theta(n)$$

# Common Asymptotic Notations and Their Importance

## **$O(1)$ - Constant Time**

- An algorithm with constant time complexity has a running time that does not change with the size of the input. It takes the same amount of time to execute regardless of the input size.
- This is the most efficient time complexity. Operations like accessing an element in an array by index or performing a basic arithmetic operation are  $O(1)$ .

## **$O(\log n)$ - Logarithmic Time**

- An algorithm with logarithmic time complexity has a running time that grows logarithmically with the input size. It is efficient for large inputs.
- Algorithms that divide the problem size in each step, like binary search, have logarithmic time complexity.

# Common Asymptotic Notations and Their Importance

## **$O(n)$ - Linear Time**

- An algorithm with linear time complexity has a running time that grows linearly with the input size.
- Algorithms that iterate through all elements of the input, like finding the maximum value in an array, have linear time complexity.

## **$O(n \log n)$ - Linearithmic Time**

- An algorithm with Linearithmic time complexity has a running time that grows in proportion to  $n \log n$ . It is more efficient than quadratic time but less efficient than linear time for large inputs.
- Efficient sorting algorithms like merge sort and quicksort have Linearithmic time complexity.

# Common Asymptotic Notations and Their Importance

## **$O(n^2)$ - Quadratic Time**

- An algorithm with quadratic time complexity has a running time that grows quadratically with the input size.
- Algorithms with nested loops that process each pair of input elements, like bubble sort, have quadratic time complexity.

## **$O(2^n)$ - Exponential Time**

- An algorithm with exponential time complexity has a running time that grows exponentially with the input size.
- Algorithms that solve problems by recursively exploring all possibilities, like the recursive solution for the Fibonacci sequence, have exponential time complexity.

# Common Asymptotic Notations and Their Importance

## **$O(n!)$ - Factorial Time**

- An algorithm with factorial time complexity has a running time that grows factorially with the input size.
- Algorithms that generate all permutations of an input, like solving the traveling salesman problem by brute force, have factorial time complexity.



# Common Asymptotic Notations and Their Importance

| Notation      | Description       | Example                    |
|---------------|-------------------|----------------------------|
| $O(1)$        | Constant time     | Accessing an array element |
| $O(\log n)$   | Logarithmic time  | Binary search              |
| $O(n)$        | Linear time       | Finding max in an array    |
| $O(n \log n)$ | Linearithmic time | Merge sort                 |
| $O(n^2)$      | Quadratic time    | Bubble sort                |
| $O(2^n)$      | Exponential time  | Recursive Fibonacci        |
| $O(n!)$       | Factorial time    | Generating permutations    |

# Recurrence Relation

A recurrence relation is a mathematical expression that defines a sequence in terms of its previous terms. In the context of algorithmic analysis, it is often used to model the time complexity of recursive algorithms.

General form of a Recurrence Relation:

$$a_n = f(a_{n-1}, a_{n-2}, \dots, a_{n-k})$$

where **f** is a function that defines the relationship between the current term and the previous terms.

There are mainly three ways of solving recurrences:

- Master Theorem
- Substitution Method
- Recurrence Tree Method

# Master Theorem

The master method is a formula for solving recurrence relations of the form:

$$T(n) = aT(n/b) + f(n),$$

where,

$n$  = size of input

$a$  = number of subproblems in the recursion

$n/b$  = size of each subproblem. All subproblems are assumed to have the same size.

$f(n)$  = cost of the work done outside the recursive call, which includes the cost of dividing the problem and cost of merging the solutions

Here,  $a \geq 1$  and  $b > 1$  are constants, and  $f(n)$  is an asymptotically positive function.

# Master Theorem

Let  $T(n)$  be a monotonically increasing function that satisfies:

$$T(n) = a T(n/b) + f(n)$$

$$T(1) = c$$

where  $a \geq 1$ ,  $b \geq 2$ ,  $c > 0$ . If  $f(n)$  is  $\Theta(n^d)$  where  $d \geq 0$  then

$$T(n) = \begin{cases} \Theta(n^d), & \text{if } a < b^d \\ \Theta(n^d \log n), & \text{if } a = b^d \\ \Theta(n^{\log_b a}), & \text{if } a > b^d \end{cases}$$

# Master Theorem: Example 1

- 

Let  $T(n) = T(n/2) + \frac{1}{2} n^2 + n$ . What are the parameters?

$$a = 1$$

$$b = 2$$

$$d = 2$$

Therefore, which condition applies?

$1 < 2^2$ , case 1 applies

We conclude that,  $T(n) \in \Theta(n^d) = \Theta(n^2)$

# Master Theorem: Example 2

- 

Let  $T(n) = 2 T(n/4) + \sqrt{n} + 42$ . What are the parameters?

$$a = 2$$

$$b = 4$$

$$d = 1/2$$

Therefore, which condition applies?

$$2 = 4^{1/2}, \text{ case 2 applies}$$

We conclude that,  $T(n) \in \Theta(n^d \log n) = \Theta(\log n \sqrt{n})$

# Master Theorem: Example 3

Let  $T(n) = 3 T(n/2) + 3/4n + 1$ . What are the parameters?

$$a = 3$$

$$b = 2$$

$$d = 1$$

Therefore, which condition applies?

$3 > 2^1$ , case 3 applies

We conclude that,  $T(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 3})$

# Master Theorem: Pitfalls

You cannot use the Master Theorem if

- $T(n)$  is not monotone, e.g.  $T(n) = \sin(x)$
- $f(n)$  is not a polynomial, e.g.,  $T(n) = 2T(n/2) + 2^n$
- $b$  cannot be expressed as a constant, e.g.  $T(n) = T(\sqrt{n})$



# More Examples of Master's Theorem

- $T(n) = 3T(n/5) + n$
- $T(n) = 2T(n/2) + n$
- $T(n) = 2T(n/2) + 1$
- $T(n) = T(n/2) + n$
- $T(n) = T(n/2) + 1$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < \underline{b^d} \\ \Theta(n^d \log n) & \text{if } a = \underline{b^d} \\ \Theta(n^{\log_b a}) & \text{if } a > \underline{b^d} \end{cases}$$

where  $a \geq 1$ ,  $b \geq 2$ ,  $c > 0$ . If  $f(n)$  is  $\Theta(n^d)$  where  $d \geq 0$

# Back Substitution Method

- Analyzing the running time of a recursive algorithm using the back substitution method involves expressing the time complexity of the algorithm in terms of its recursive calls and then solving the recurrence relation.
- In back substitution method, the goal is to express recurrence relation  $T(n)$  in terms of simpler subproblems until you reach a base case.
- The base case is the smallest instances of the problem for which you know the running time directly.
- For example, if your algorithm terminates when  $n = 1$ , then  $T(1)$  is your base case.

# Back Substitution Method Example

- Recurrence relation:  $T(n) = 2 * T(n/2) + n$
- Base case:  $T(1) = 1$  (assuming the algorithm terminates when  $n = 1$ )
- Back substitution method solve this recurrence as follows:
  - $T(n) = 2 * T(n/2) + n$
  - $T(n) = 2 * [2 * T(n/4) + n/2] + n$
  - $T(n) = 2^2 * T(n/2^2) + 2 * n/2 + n$
  - $T(n) = 2^3 * T(n/2^3) + 2^2 * n/2^2 + 2 * n/2 + n$
- Continue this process until you reach the base case  $T(1)$ , then simplify and solve the equation to find the time complexity. In this case, it would lead to a time complexity of  $O(n * \log n)$  for many divide-and-conquer algorithms.

# Back Substitution Method Example - 1

$$\begin{array}{ll} T(n) = 1 + T(n-1) & n > 1 \\ = 1 & n = 1 \end{array}$$

$$\rightarrow T(n) = 1 + T(n-1)$$

$$\Rightarrow T(n-1) = 1 + T(n-2)$$

$$\Rightarrow T(n-2) = 1 + T(n-3)$$

$$\Rightarrow T(n) = 1 + [1 + T(n-2)] = 2 + T(n-2) = 2 + [1 + T(n-3)] = 3 + T(n-3) = \dots = K + T(n-k)$$

It will stop at  $T(1) \Rightarrow T(n) \Rightarrow T(n-k)$

$$\rightarrow n - k = 1 \Rightarrow k = n - 1$$

$$T(n) = n - 1 + 1 = n \Rightarrow O(n)$$

# Back Substitution Method Example - 2

$$T(n) = n + T(n-1) \quad n > 1$$
$$= 1 \quad n = 1$$

$$T(n-1) = (n-1) + T(n-2)$$

$$T(n-2) = (n-2) + T(n-3)$$

$$T(n-1) = n + T(n-1) = n + [(n-1) + T(n-2)]$$
$$= n + (n-1) + [(n-2) + T(n-3)]$$

$$T(n) = n + (n-1) + (n-2) + T(n-3)$$

.

.

$$T(n) = n + (n-1) + (n-2) + \dots + (n-k+1) + T(n-k)$$

It will stop at  $T(1) \Rightarrow T(n) \Rightarrow T(n-k)$

$$n-k=1 \Rightarrow k=n-1$$

$$\Rightarrow T(n) = n + (n-1) + (n-2) + \dots + n - (n-1) + 1 + T(1)$$

$$\Rightarrow = n + (n-1) + (n-2) + \dots + 2 + 1 = \frac{n(n+1)}{2} = O(n^2)$$

# Back Substitution Method Example - 3

$$\begin{array}{ll} T(n) = 1 & n=0 \\ = T(n-1) + \log n & n>1 \end{array}$$

$$T(n) = T(n-1) + \log n$$

$$T(n-1) = T(n-2) + \log(n-1)$$

$$T(n-2) = T(n-3) + \log(n-2)$$

$$\Rightarrow T(n) = \log n + T(n-1) = \log n + [\log(n-1) + T(n-2)]$$

$$\Rightarrow = \log n + \log(n-1) + \log(n-2) + T(n-3)$$

.

.

$$= \log n + \log(n-1) + \dots + \log(n-(k-1)) + T(n-k)$$

$$= [\log n + \log(n-1) + \dots + \log(1)] + 1$$

$$= \log [n * (n-1) * (n-2) * \dots * 1] + 1 = \log[n!] + 1 \Rightarrow \mathbf{O(n \log n)} \quad [\text{we can replace } n! \text{ to } n^n]$$

# Back Substitution Method Example - 4

$$T(n) = 2 T(n-1) + 1$$
$$= 1$$

$$n > 0$$
$$n = 0$$

$$T(n-1) = 2T(n-2) + 1$$

$$T(n-2) = 2T(n-3) + 1$$

$$\begin{aligned} \rightarrow T(n) &= 2 [1 + 2 T(n-2)] + 1 \\ &= 3 + 4 T(n-2) \approx (2^2 - 1) + 2^2 T(n-2) \\ &= 3 + 4 [2T(n-3) + 1] \\ &= 7 + 8T(n-3) \approx (2^3 - 1) + 2^3 T(n-3) \end{aligned}$$

.

.

$$\begin{aligned} \rightarrow (2^k - 1) + 2^k T(n-k) & \quad \text{exit condition } n - k = 0 \Rightarrow k = n \\ &= 2^k + 2^k - 1 = 0 \quad (2^n) \end{aligned}$$

$$\sum_{i=0}^k 2^i = 2^{k+1} - 1$$

# More Examples of Back Substitution Method

1.  $T(n) = 1, \quad n=1$   
 $=T(n/2) + 1, n > 1$
2.  $T(n) = 1, \quad n=1$   
 $=T(n/2) + n, n > 1$
3.  $T(n) = 1, \quad n=1$   
 $=2T(n/2) + n, n > 1$
4.  $T(n) = c, \quad n=1$   
 $=2T(n/2) + c, n > 1$
5.  $T(n) = 1 + 2T(n/2) + 1, n > 1$   
 $=1, \quad n=1$



# Recursion Tree | Solving Recurrence Relations

A recursion tree is a tree where each node represents the cost of a certain recursive sub-problem.

We sum up the values in each node to get the cost of the entire algorithm.

Steps to Solve Recurrence Relations Using Recursion Tree Method-

1. Draw a recursion tree based on the given recurrence relation.

2. Determine-

- Cost of each level
- Total number of levels in the recursion tree
- Number of nodes in the last level
- Cost of the last level

3. Add cost of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation.

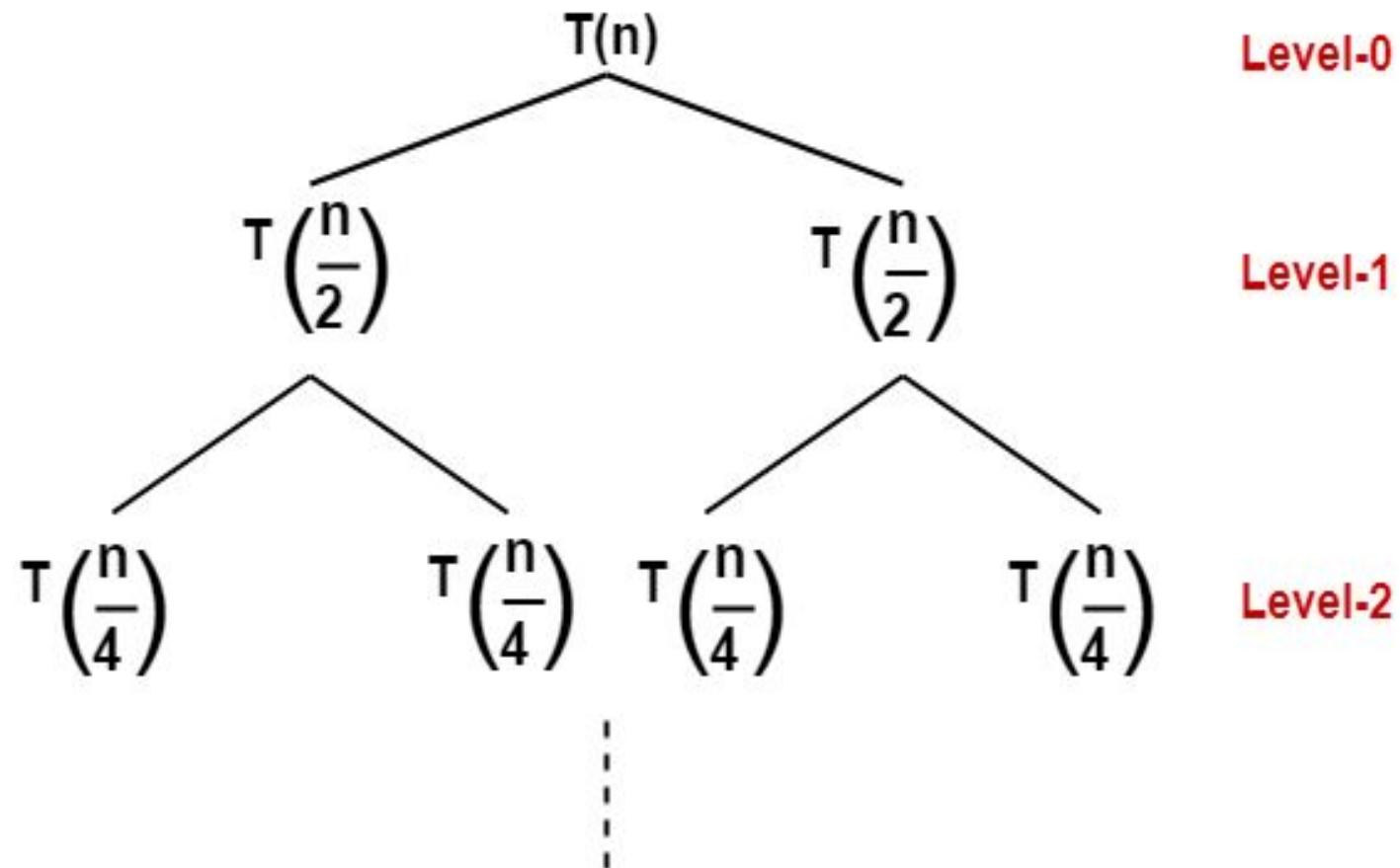
## PRACTICE PROBLEMS BASED ON RECURSION TREE

Problem-01: Solve the following recurrence relation using recursion tree method -  $T(n) = 2T(n/2) + n$

Step-01: Draw a recursion tree based on the given recurrence relation. The given recurrence relation shows-

- A problem of size  $n$  will get divided into 2 sub-problems of size  $n/2$ .
- Then, each sub-problem of size  $n/2$  will get divided into 2 sub-problems of size  $n/4$  and so on.
- At the bottom most layer, the size of sub-problems will reduce to 1.

This is illustrated through following recursion tree-

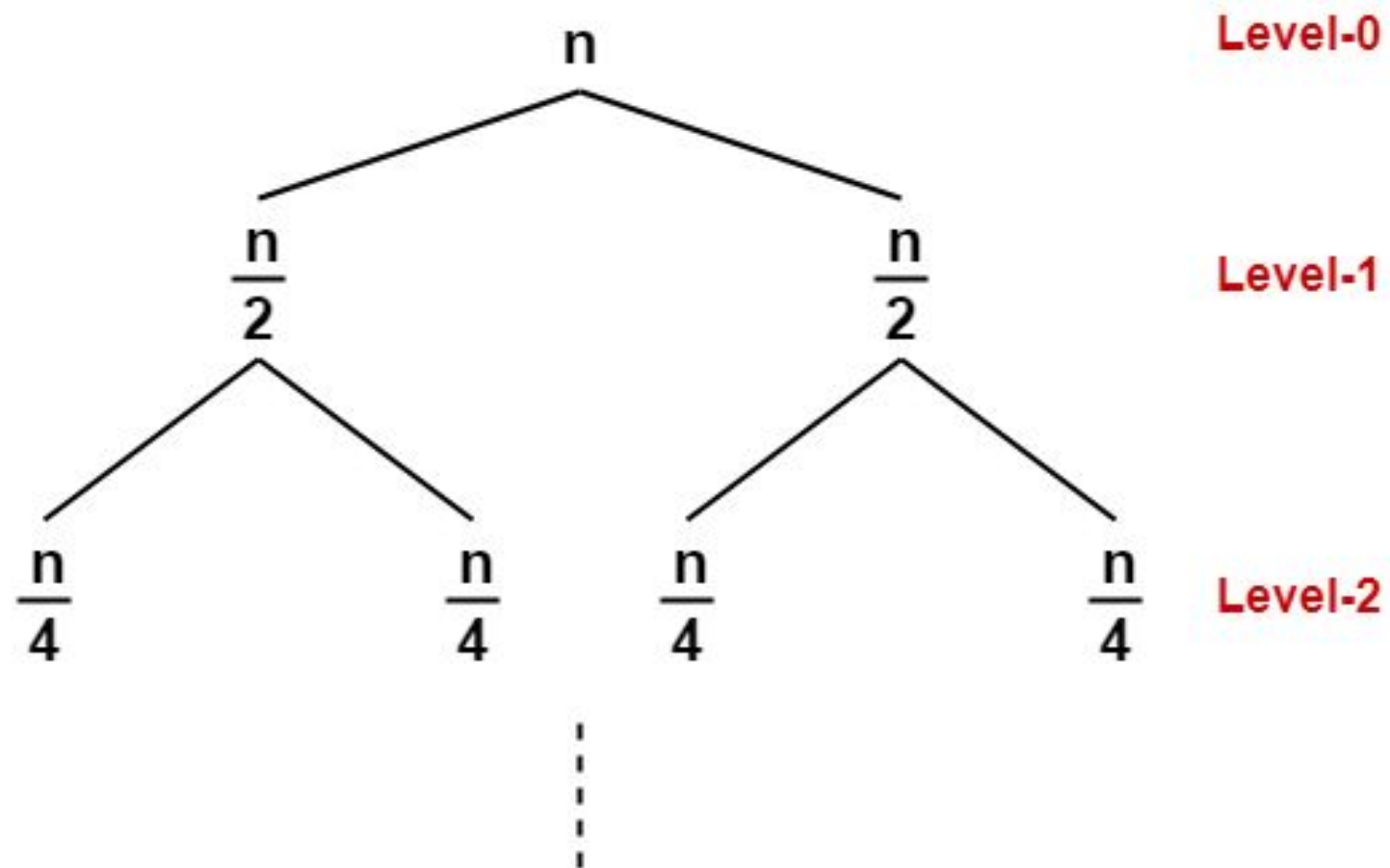


# The given recurrence relation shows-

The cost of dividing a problem of size  $n$  into its 2 sub-problems and then combining its solution is  $n$ .

The cost of dividing a problem of size  $n/2$  into its 2 sub-problems and then combining its solution is  $n/2$  and so on.

**This is illustrated through following recursion tree where each node represents the cost of the corresponding sub-problem-**



## Step-02:

Determine cost of each level-

Cost of level-0 =  $n$

Cost of level-1 =  $n/2 + n/2 = n$

Cost of level-2 =  $n/4 + n/4 + n/4 + n/4 = n$  and so on.

# Step-03

- **Determine total number of levels in the recursion tree-**

Size of sub-problem at level-0 =  $n/2^0$

Size of sub-problem at level-1 =  $n/2^1$

Size of sub-problem at level-2 =  $n/2^2$

Continuing in similar manner, we have-

Size of sub-problem at level-i =  $n/2^i$

Suppose at level-x (last level), size of sub-problem becomes 1. Then-

$$n / 2^x = 1$$

$$2^x = n$$

Taking log on both sides, we get-

$$x \log 2 = \log n$$

$$x = \log_2 n$$

∴ Total number of levels in the recursion tree =  $\log_2 n + 1$

# Step-04

Determine number of nodes in the last level-

Level-0 has  $2^0$  nodes i.e. 1 node

Level-1 has  $2^1$  nodes i.e. 2 nodes

Level-2 has  $2^2$  nodes i.e. 4 nodes

Continuing in similar manner, we have-

Level- $\log_2 n$  has  $2^{\log_2 n}$  nodes i.e.  $n$  nodes



# Step-05 & Step-06

## Step-05:

Determine cost of last level-

$$\text{Cost of last level} = n \times T(1) = \theta(n)$$

## Step-06:

Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation

$$T(n) = \underbrace{\{ n + n + n + \dots \}}_{\text{For } \log_2 n \text{ levels}} + \theta(n)$$

For  $\log_2 n$  levels

$$= n \times \log_2 n + \theta(n)$$

$$= n \log_2 n + \theta(n)$$

$$= \theta(n \log_2 n)$$

# Class Work on Recursion Tree

Solve the following recurrence relation using recursion tree method-

- $T(n) = T(n/5) + T(4n/5) + n$
- $T(n) = 3T(n/4) + cn^2$

# Suggestive Questions

## 1. Asymptotic Notations

Q1. Define Big-O, Big-Theta, and Big-Omega notations. Explain the difference with the help of graphs and examples.

Q2. Prove that  $f(n) = 3n^2 + 10n + 5$  is  $\Theta(n^2)$ .

Q3. If  $f(n) = O(g(n))$  and  $g(n) = O(h(n))$ , is it necessarily true that  $f(n) = O(h(n))$ ? Justify.

Q4. Discuss the importance of asymptotic notation in algorithm analysis.

Q5. Why is it often acceptable to ignore lower-order terms and constants in asymptotic notation?

Q6. Arrange the following functions in increasing order of growth:  $\log n$ ,  $\sqrt{n}$ ,  $n$ ,  $n \log n$ ,  $n^2$ ,  $2^n$ ,  $n!$

Q7. Prove  $\log(n!) = \Theta(n \log n)$

# Suggestive Questions

## 2. Recurrence Relations

Q8. Define recurrence relation. Give real-world or algorithmic examples where recurrence relations naturally arise.

Q9. What are the different methods of solving recurrence relations? Compare their advantages and limitations.

Q10. How does recursion relate to recurrence relations in algorithms?

Q11. Solve the recurrence:  $T(n) = T(n-1) + n$ , with  $T(1) = 1$

Q12. Solve  $T(n) = 2T(n/2) + n$ , using a) Recursion Tree, b) Master Theorem, c) Back Substitution.

# Suggestive Questions

## 3. Master Theorem

13. State and prove the Master Theorem for divide-and-conquer recurrences.

14. What are the limitations of the Master Theorem? Give an example of a recurrence it cannot solve.

15. Explain the difference between Case 1, Case 2, and Case 3 of the Master Theorem with examples.

16. Use the Master Theorem to find the time complexity of:

a)  $T(n)=4T(n/2)+n$

b)  $T(n)=2T(n/2)+n\log n$

c)  $T(n)=T(n/2)+1$

17. Identify whether the Master Theorem can be applied to:

$T(n)=2T(n/2)+\frac{n}{n\log n}$ . If not, suggest another method.

# Suggestive Questions

## 4. Back Substitution Method

18. Explain the back substitution method for solving recurrence relations.

19. When is back substitution preferable over Master Theorem?

20. Solve using back substitution:  $T(n) = T(n-1) + 3$ , with  $T(1) = 2$

21. Solve:  $T(n) = 2T(n/2) + n$ , with  $T(1) = 1$

22. Derive a closed-form expression for:  $T(n) = T(n-1) + 2n - 1$ ,  $T(1) = 1$

# Suggestive Questions

## 5. Recursion Tree Method

23. Describe the recursion tree method and its use in solving recurrence relations.

24. What is the intuition behind comparing work done at each level of the tree?

25. Use recursion tree method to solve:

a)  $T(n)=3T(n/4)+n$

b)  $T(n)=2T(n/2)+n\log n$

26. For the recurrence  $T(n)=T(n/2)+n$ , draw the recursion tree and find the total work at each level.